FACULDADE DE NOVA SERRANA – FANS
ENGENHARIA DE SOFTWARE
DISCIPLINA: COMPILADORES



Mateus Artur
Rafael Araújo
Otávio de Pádua

PORTULANG

Documento final: especificação da linguagem e documentação técnica

Nova Serrana – MG
2026

1. Apresentação

A PortuLang é uma linguagem de programação imperativa e pequena, criada para a disciplina de Compiladores. Seu objetivo é ser fácil de ler e de aprender: todas as palavras-chave são em português e toda variável precisa ter o tipo declarado no momento em que é criada. Os programas são arquivos de texto com a extensão **.pl**.

O projeto implementa o *front-end* de um compilador (análises léxica, sintática e semântica) e uma etapa de **geração de código** que traduz o programa para Python. A linguagem é, portanto, **híbrida**: é compilada para Python (um arquivo .py equivalente) e depois interpretada pelo Python. Não há geração de código de máquina. Um interpretador próprio, que percorre a árvore sintática, também executa os programas e alimenta a execução passo a passo do Studio.

Pelo tamanho e pela clareza das mensagens de erro, a PortuLang é boa para **aprendizado** (de quem está começando a programar e de quem está estudando compiladores) e para **demonstração**: o PortuLang Studio mostra, em uma cena 3D, o que cada fase faz com o código.

1.1 Princípios de projeto

- **Português claro:** se, senao, enquanto, escrever, ler.
- **Tipagem explícita e estática:** o tipo vem antes do nome, e erros de tipo são detectados antes de o programa executar.
- **Poucas regras:** blocos com { }, comandos terminados em ;, sem indentação significativa.
- **Erros amigáveis:** toda mensagem informa linha, coluna e uma explicação em português.

1.2 Mapa dos requisitos obrigatórios

Requisito do projeto	Como a PortuLang atende
Declaração de variáveis	inteiro, decimal (ponto flutuante), texto e logico (booleano), com tipo obrigatório
Atribuição	x = expressao;, com checagem de tipo
Operadores aritméticos	+ - * / (e + para concatenar texto)
Operadores relacionais	> < == != e, além deles, >= <=
Condicionais	se (...) { } senao { }, com senao se encadeado
Repetição	enquanto (...) { }
Entrada / saída	ler(x); e escrever(...);
Análise léxica / sintática / semântica	Seções 4, 5, 7 e 8; implementadas em lexer.py, parser.py e semantico.py
Execução (tradutor funcional)	Compilação para Python e interpretador sobre a árvore (seções 9 e 10)

2. Estrutura de um programa

Todo programa começa com **programa** e termina com **fim**. Entre os dois ficam os comandos, executados de cima para baixo.

```
programa
    inteiro idade = 20;
    inteiro limite = 18;
    se (idade >= limite) {
        escrever("Maior de idade");
    } senao {
        escrever("Menor de idade");
    }
    inteiro contador = 1;
    enquanto (contador <= 5) {
        escrever(contador);
        contador = contador + 1;
    }
fim
```

Comentários começam com // e vão até o fim da linha. Espaços, tabulações e quebras de linha são ignorados pelo analisador léxico.

3. Tipos, variáveis e escopo

Tipo	Valores	Exemplo de literal	Valor inicial se não for dado
`inteiro`	números inteiros	`42, 0, 1500`	0
`decimal`	números com parte fracionária	`3.14, 0.5`	0.0
`texto`	sequência de caracteres	`"Olá, mundo"`	"" (texto vazio)
`logico`	verdadeiro ou falso (booleano)	`verdadeiro, falso`	falso

O tipo decimal equivale ao float/double de outras linguagens (ponto flutuante de precisão dupla, 64 bits) e o logico equivale ao bool.

Declaração. O formato é `tipo nome;` ou `tipo nome = expressao;`. A declaração é obrigatória antes do primeiro uso.

```
inteiro quantidade;
decimal preco = 9.90;
texto nome = "Ana";
logico ativo = verdadeiro;
```

Compatibilidade de tipos.

- Um inteiro pode ser guardado em um decimal (conversão automática e sem perda).
- Um decimal não pode ser guardado em um inteiro (erro semântico).
- texto só é compatível com texto, e logico só com logico.
- Aritmética entre inteiro e decimal resulta em decimal. A divisão / entre dois inteiros resulta em inteiro (divisão inteira, truncada em direção ao zero).
- + entre dois textos concatena; nenhum outro operador aritmético aceita texto.

Valores lógicos. O resultado de comparações e dos operadores e / ou / nao é do tipo logico e pode ser guardado em variáveis. As condições de se e enquanto exigem uma expressão logico. Os operadores > < >= <= só valem para números; == e != valem para dois operandos do mesmo tipo (um inteiro pode ser comparado a um decimal).

Escopo. Cada bloco { } cria um escopo. Uma variável vale do ponto onde foi declarada até o fim do bloco. Para evitar confusão, não é permitido declarar um nome que já está visível (nem em

escopo interno).

4. Análise léxica: tokens e expressões regulares

4.1 Palavras-chave (reservadas)

Token	Lexema	Função
PROGRAMA	programa	início do programa
FIM	fim	fim do programa
TIPO_INTEIRO	inteiro	tipo de variável
TIPO_DECIMAL	decimal	tipo de variável
TIPO_TEXTO	texto	tipo de variável
TIPO_LOGICO	logico	tipo de variável
VERDADEIRO	verdadeiro	literal lógico
FALSO	falso	literal lógico
SE	se	condicional
SENAO	senao	alternativa do condicional
ENQUANTO	enquanto	repetição
LER	ler	entrada de dados
ESCREVER	escrever	saída de dados
E	e	conjunção lógica
OU	ou	disjunção lógica
NAO	nao	negação lógica

As palavras-chave são escritas em minúsculas e **sem acentos** (senao, nao), para evitar problemas de teclado e de codificação.

4.2 Demais tokens e expressões regulares

Token	Expressão regular	Exemplos
ID	[A-Za-z_][A-Za-z0-9_]* (que não seja palavra-chave)	idade, total_1, _x
LIT_INTEIRO	[0-9]+	0, 42, 1500
LIT_DECIMAL	[0-9]+\.[0-9]+	3.14, 0.5
LIT_TEXTO	"([^"\\\n]|\\.)*"	"oi", "linha\n"
MAIS / MENOS	\+ | -	+ -
VEZES / DIVIDE	* | /	* /
IGUAL_IGUAL / DIFERENTE	== | !=	== !=
MENOR / MAIOR	< | >	< >
MENOR_IGUAL / MAIOR_IGUAL	<= | >=	<= >=
ATRIBUI	=	=
ABRE_PAR / FECHA_PAR	\(| \)	( )
ABRE_CHAVE / FECHA_CHAVE	\{ | \}	{ }
PONTO_VIRGULA / VIRGULA	; | ,	; ,
COMENTARIO (ignorado)	//[^\n]*	// texto
ESPACO (ignorado)	[\t\r\n]+	

Regras do lexer. (1) Vale sempre o maior casamento: >= é um único token, não dois. (2) Um identificador é comparado com a tabela de palavras-chave antes de virar ID. (3) Dentro de textos são aceitas as sequências \n, \t, \" e \\. (4) Cada token guarda tipo, lexema, linha e coluna.

Erros léxicos: caractere inválido (por exemplo @), texto sem aspas de fechamento, número decimal malformado (3.) e sequência de escape inválida.

5. Gramática formal (EBNF)

Notação: { x } = zero ou mais; [x] = opcional; | = alternativa; palavras em MAIÚSCULAS são tokens da seção 4.

```
programa     = PROGRAMA , { comando } , FIM ;
comando      = declaracao | atribuicao | leitura
             | escrita | condicional | repeticao ;
declaracao   = tipo , ID , [ ATRIBUI , expressao ] , PONTO_VIRGULA ;
tipo         = TIPO_INTEIRO | TIPO_DECIMAL | TIPO_TEXTO | TIPO_LOGICO ;
atribuicao   = ID , ATRIBUI , expressao , PONTO_VIRGULA ;
leitura      = LER , ABRE_PAR , ID , FECHA_PAR , PONTO_VIRGULA ;
escrita      = ESCREVER , ABRE_PAR , expressao ,
               { VIRGULA , expressao } , FECHA_PAR , PONTO_VIRGULA ;
condicional  = SE , ABRE_PAR , expressao , FECHA_PAR , bloco ,
               [ SENAO , ( bloco | condicional ) ] ;
repeticao    = ENQUANTO , ABRE_PAR , expressao , FECHA_PAR , bloco ;
bloco        = ABRE_CHAVE , { comando } , FECHA_CHAVE ;

(* expressoes, da MENOR para a MAIOR precedencia *)
expressao    = disjuncao ;
disjuncao    = conjuncao    , { OU , conjuncao } ;
conjuncao    = igualdade    , { E  , igualdade } ;
igualdade    = relacional   , { ( IGUAL_IGUAL | DIFERENTE ) , relacional } ;
relacional   = soma         , { ( MENOR | MAIOR | MENOR_IGUAL | MAIOR_IGUAL ) , soma } ;
soma         = termo        , { ( MAIS | MENOS ) , termo } ;
termo        = unario       , { ( VEZES | DIVIDE ) , unario } ;
unario       = ( MENOS | NAO ) , unario | primario ;
primario     = LIT_INTEIRO | LIT_DECIMAL | LIT_TEXTO | ID
             | VERDADEIRO | FALSO
             | ABRE_PAR , expressao , FECHA_PAR ;
```

5.1 Precedência e associatividade

Nível (maior para menor)	Operadores	Associatividade
1	parênteses ( )	-
2	unários: - nao	direita
3	* /	esquerda
4	+ -	esquerda
5	< > <= >=	esquerda
6	== !=	esquerda
7	e	esquerda
8	ou	esquerda

A gramática é LL(1) depois de fatorada, o que permite implementar o analisador sintático como descendente recursivo, com uma função por regra. O *dangling else* é resolvido ligando o senao ao se mais próximo. Como os blocos são sempre delimitados por chaves, não há ambiguidade.

6. Comandos de entrada e saída

`escrever(a, b, c);` avalia cada expressão, junta os valores sem separador e termina com quebra de linha. Para inserir espaço, use texto: `escrever("Total: ", total);`. Valores lógicos aparecem como verdadeiro ou falso.

`ler(x);` lê uma linha da entrada e converte para o tipo de x. Se o valor digitado não puder ser convertido (por exemplo, letras para um inteiro), ocorre um erro de execução. Para um logico, são aceitos verdadeiro e falso. Em decimal, a vírgula também é aceita (7,5). O comando ler só aceita uma variável já declarada.

7. Análise semântica

O validador percorre a árvore sintática com uma tabela de símbolos (pilha de escopos; cada entrada guarda nome, tipo e linha da declaração) e verifica as regras abaixo antes de qualquer execução.

Código	Regra	Exemplo que gera o erro
S01	Variável usada sem ter sido declarada	`total = 5;`
S02	Variável declarada duas vezes (nome já visível)	`inteiro a; inteiro a;`
S03	Tipo do valor incompatível com o da variável	`inteiro x = "oi";`
S04	Decimal atribuído a inteiro (perda de dados)	`inteiro x = 2.5;`
S05	Operador aplicado a tipos inválidos	`"a" - 1`
S06	Comparação entre tipos incompatíveis (ou > < fora de números)	`se (nome == 3)`
S07	Condição de se/enquanto não é logico	`enquanto (contador)`
S08	Operando de nao / e / ou não é logico	`nao 5`
S09	Leitura em variável não declarada	`ler(y);`

O compilador não para no primeiro erro semântico: acumula todos e exibe a lista completa. Formato das mensagens: `[Erro semântico S03] linha 2, coluna 21: não é possível guardar um 'texto' em 'idade', que é do tipo 'inteiro'.`

8. Catálogo de erros e recuperação

Todo erro tem um código (letra da fase + número), linha, coluna e mensagem em português. A letra indica a fase: L (léxico), P (sintático), S (semântico) e E (execução).

Código	Fase	Significado
L01	léxico	caractere inválido
L02	léxico	texto sem aspas de fechamento
L03	léxico	número decimal malformado
L04	léxico	sequência de escape inválida
P01	sintático	token esperado não encontrado
P02	sintático	expressão esperada
P03	sintático	comando inválido
P04	sintático	código depois de 'fim'
P05	sintático	parêntese ou chave aberto e nunca fechado
S01	semântico	variável não declarada
S02	semântico	variável já declarada
S03	semântico	tipo incompatível

Código	Fase	Significado
S04	semântico	decimal em inteiro (perda de dados)
S05	semântico	operador com tipos inválidos
S06	semântico	comparação inválida
S07	semântico	condição não é logico
S08	semântico	operando não é logico
S09	semântico	leitura em variável não declarada
E01	execução	divisão por zero
E02	execução	valor lido não converte para o tipo
E03	execução	limite de passos excedido
E04	execução	entrada de dados esgotada

Recuperação de erros. O analisador sintático não para no primeiro erro: ao falhar, ele registra o problema e sincroniza (avança até o próximo ; ou }, ou até o início do próximo comando) para continuar e relatar os demais. Os trechos que não entendeu entram na árvore como nós de erro, de modo que é possível mostrar a **árvore parcial**, marcando onde ela quebrou. Quando falta fechar um (ou um {, o erro P05 aponta o símbolo que ficou aberto, e não só o lugar onde o analisador percebeu o problema.

A entrada esgotada (E04) não é tratada como falha no Studio: o programa simplesmente para e espera o usuário digitar o próximo valor. Um limite de passos (E03) protege contra laços infinitos.

9. Arquitetura e implementação

O pipeline segue a arquitetura sugerida na proposta da disciplina e acrescenta a geração de código:



Fase	Entrada	Saída	Módulo
Análise léxica	código-fonte	lista de tokens	lexer.py
Análise sintática	tokens	AST (ou AST parcial, com erros)	parser.py, ast_nodes.py
Análise semântica	AST	AST anotada com tipos + tabela de símbolos	semantico.py
Geração de código	AST validada	programa Python (e código de pilha)	transpilador.py, codigo.py
Execução	programa Python / AST	saída do programa	Python; interpretador.py

Tecnologia. O núcleo é escrito em Python 3, somente com a biblioteca padrão. Não foi usado ANTLR: o lexer e o parser descendente recursivo são escritos à mão, para que toda a equipe domine e consiga explicar o código. A interface do Studio usa HTML, CSS e JavaScript, com

three.js na cena 3D.

Ligação entre as fases. Cada token guarda linha e coluna, e cada nó da AST guarda a faixa de tokens que cobre (campos ti e tf). É isso que permite ao Studio acender, no código, o trecho correspondente a um nó da árvore, a uma instrução gerada ou a um passo da execução. O interpretador aceita um *observador*, chamado antes de cada comando com as variáveis visíveis, que é a base da execução passo a passo.

9.1 Módulos e tamanho

Arquivo	Linhas (sem brancas)	Responsabilidade
`lexer.py`	124	Análise léxica: texto -> lista de tokens
`tokens.py`	52	Definição dos tipos de token e das palavras-chave
`parser.py`	238	Análise sintática descendente recursiva, com recuperação de erro
`ast_nodes.py`	105	Nós da árvore sintática (AST) e sua conversão para JSON
`semantico.py`	145	Validação de declarações e tipos; tabela de símbolos
`transpilador.py`	123	Geração de código: AST -> programa Python
`codigo.py`	183	Código de máquina de pilha e máquina virtual (conferência)
`interpretador.py`	156	Interpretador que percorre a AST; execução passo a passo
`erros.py`	40	Classe de erro e catálogo de códigos
`server.py`	241	Servidor local do Studio (API e arquivos)
`app.js / scene3d.js`	1721	Interface do Studio e cena 3D (three.js)
`desktop.py`	33	Aplicativo de janela própria (Windows)

10. Geração de código e execução

10.1 Compilação para Python

Depois de validado, o programa é traduzido para um programa Python equivalente. A tradução é feita em uma passada sobre a AST, comando por comando:

PortuLang	Python gerado
`inteiro x = 5;`	`x = 5`
`decimal d = 7;`	`d = float(7)`
`x = a + b;`	`x = a + b (decimal: x = float(a + b))`
`ler(x);`	`x = _ler("inteiro", "x")`
`escrever(a, b);`	`_escrever(a, b)`
`se (c) { } senao se (d) { } senao { }`	`if c: ... / elif d: ... / else: ...`
`enquanto (c) { }`	`while c:`

PortuLang	Python gerado
`a e b / a ou b / nao a`	`a and b / a or b / not a`
`a / b`	`_dividir(a, b) (divisão inteira truncada e erro em zero)`

Quatro funções auxiliares ficam no início do arquivo gerado (`_fmt`, `_escrever`, `_dividir` e `_ler`) e cuidam do que a PortuLang faz diferente do Python: mostrar verdadeiro/falso, dividir inteiros, aceitar vírgula em decimais e converter a entrada para o tipo da variável. Nomes de variáveis que coincidem com palavras do Python (class, print...) ganham um _ no final. O arquivo gerado roda sozinho com `python programa.py`.

Exemplo: o programa da seção 11.1 (exemplo 02_media) gera o seguinte código (sem as funções auxiliares):

```
nome = ""
nota1 = 0.0
nota2 = 0.0
_escrever('Qual o seu nome? ')
nome = _ler("texto", "nome")
_escrever('Nota 1: ')
nota1 = _ler("decimal", "nota1")
_escrever('Nota 2: ')
nota2 = _ler("decimal", "nota2")
media = float(_dividir(nota1 + nota2, 2))
if media >= 7:
    _escrever(nome, ' foi aprovado(a) com ', media)
elif media >= 5:
    _escrever(nome, ' esta de recuperacao com ', media)
else:
    _escrever(nome, ' foi reprovado(a) com ', media)
```

10.2 Interpretador sobre a árvore

O interpretador percorre a AST validada, guarda as variáveis numa pilha de escopos (cada bloco abre um escopo novo) e executa cada comando. Ele existe por dois motivos: serve de referência de comportamento da linguagem e é ele que fornece o passo a passo do Studio (qual comando está rodando, valor guardado, texto mostrado, resultado de cada condição).

10.3 Código de máquina de pilha e máquina virtual

Além do Python, o compilador gera uma lista de instruções para uma máquina de pilha simples (EMPILHAR, CARREGAR, CRIAR, GUARDAR, SOMAR, SALTAR_SE_FALSO, ESCREVER...), mostrada no Studio. Uma máquina virtual em Python executa essa lista, e o Studio confere se o resultado é igual ao do interpretador. Por exemplo, `inteiro x = 1 + 2; escrever(x);` vira:

```
00  EMPILHAR   1
01  EMPILHAR   2
02  SOMAR
03  CRIAR      inteiro x
04  CARREGAR   x
05  ESCREVER   1
06  PARAR
```

10.4 Por que é híbrida

A PortuLang percorre o caminho completo de um compilador (léxico, sintático, semântico e geração de código) e o resultado é um arquivo .py: isso é a **compilação**. Depois, o Python **interpreta** esse arquivo (ele mesmo o converte em *bytecode* e o executa). Por isso a linguagem é híbrida: compilada para Python e depois interpretada, de forma parecida com a do Java, que compila para *bytecode* e roda numa máquina virtual. Não há otimização nem geração de código de máquina real.

11. PortuLang Studio

O Studio é a ferramenta de desenvolvimento e demonstração. Ele guia o código por seis etapas, cada uma com uma explicação em linguagem simples, uma cena 3D e ligação com o código-fonte: passar o mouse sobre um elemento da cena acende o trecho correspondente no código, e vice-versa. Cada fase roda no servidor (Python) apenas quando o usuário chega nela, e o Studio mostra o tempo real que ela levou.

- **Escrever:** editor com destaque de sintaxe e exemplos (corretos e com erros propositais); botões Abrir e Salvar para arquivos .pl; a cena mostra o caminho que o código vai percorrer.
- **Léxico:** cada peça do código vira um bloco colorido num tabuleiro; tabela de tokens.
- **Sintático:** as peças viram uma árvore; com erro, mostra a árvore parcial e marca onde quebrou.
- **Semântico:** selos de tipo em cada nó, tabela de símbolos e erros em vermelho.
- **Código:** o programa traduzido para Python (e, em outra aba, as instruções da máquina de pilha), ligado ao código-fonte; permite copiar e baixar o .py.
- **Executar:** caixas de variável, tela e um robô; o usuário digita os valores pedidos por ler() na própria tela; play, pausa, passo a passo e frases explicando cada comando.

Mouse na cena: roda = zoom; Ctrl + roda = subir e descer; arrastar = mover; botão direito ou Ctrl + arrastar = girar. Há tema claro e escuro (padrão: o do sistema) e as janelas podem ser redimensionadas ou abertas em tela cheia.

11.1 As etapas

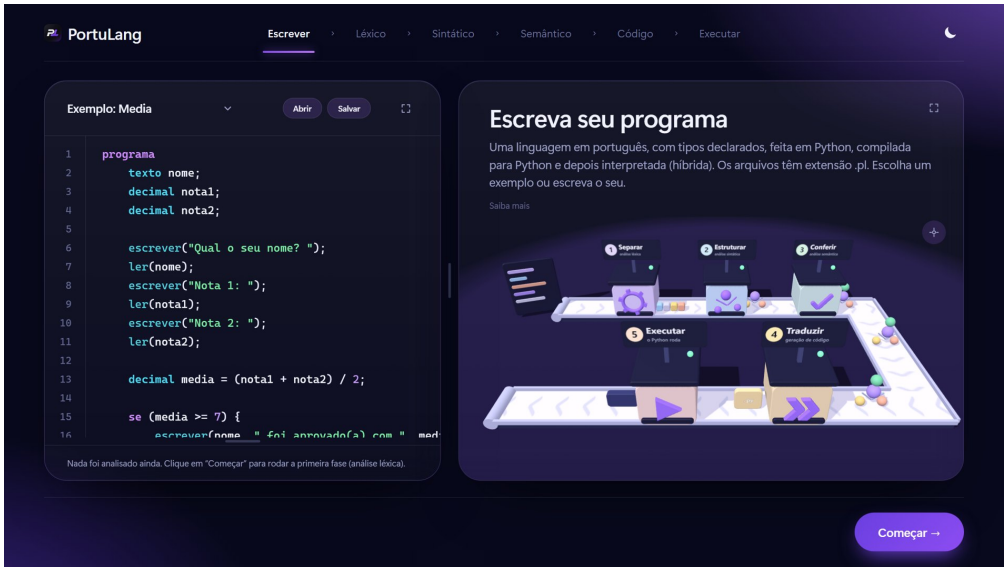


Figura 1. Etapa Escrever: o caminho do código pelas fases, com as descrições ao passar o mouse.

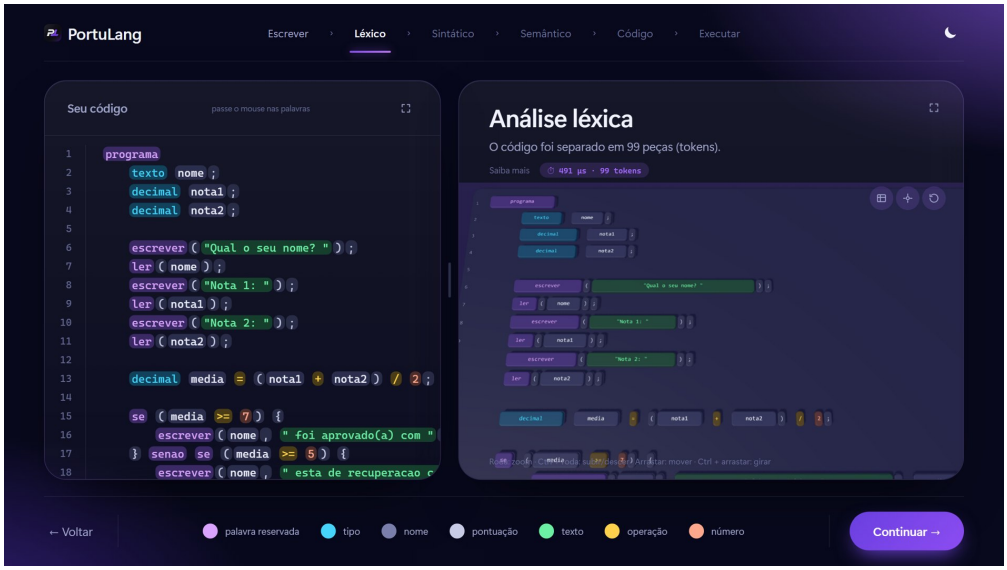


Figura 2. Etapa Léxico.

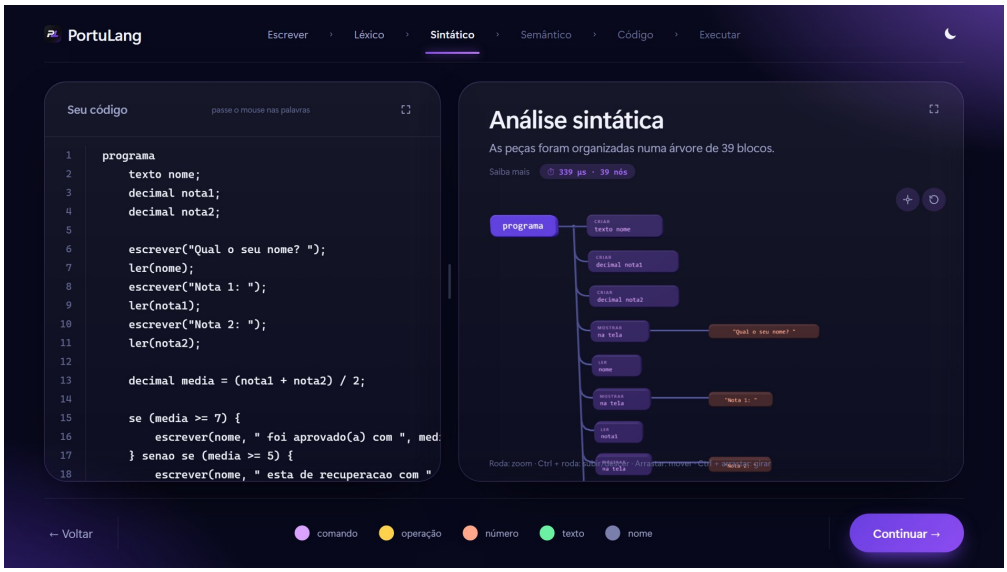


Figura 3. Etapa Sintático: a árvore sintática.

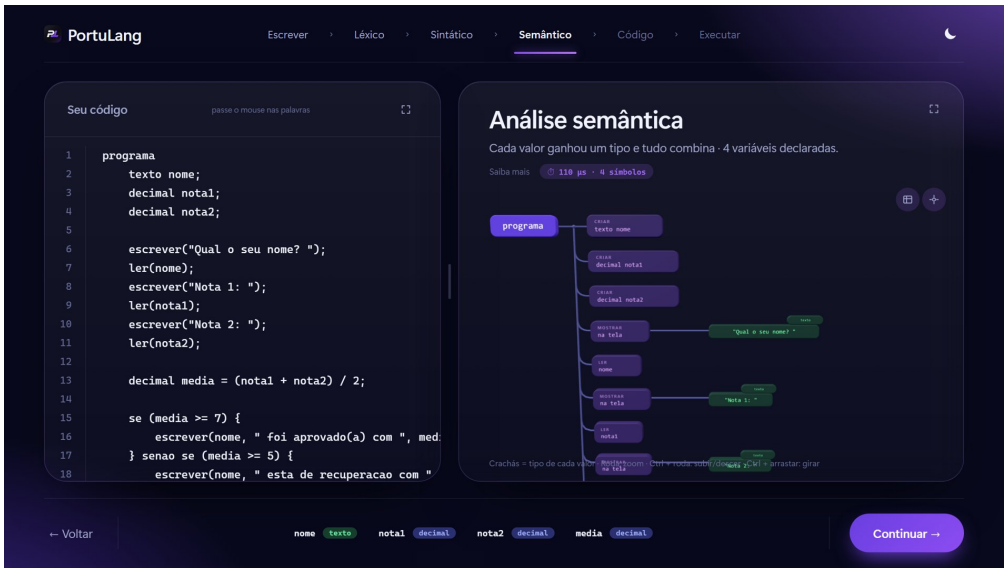


Figura 4. Etapa Semântico: selos de tipo e tabela de símbolos.

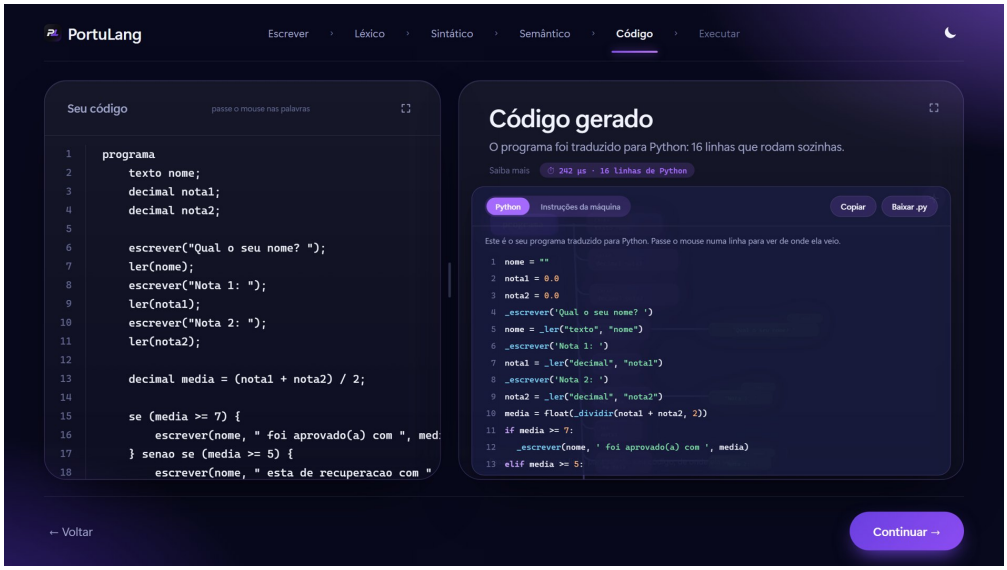


Figura 5. Etapa Código: o programa traduzido para Python.

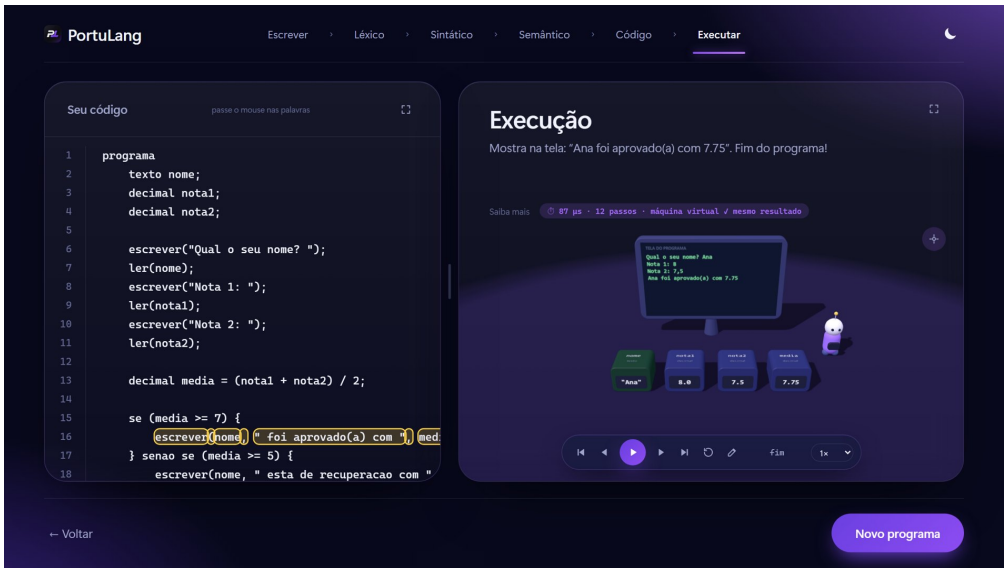


Figura 6. Etapa Executar: caixas das variáveis, tela do programa e o robô.

11.2 Erros no Studio

Se uma fase encontra erros, ela explica em português, marca o ponto no código e o botão passa a ser “Corrigir no código”. As figuras abaixo mostram a árvore parcial de um erro sintático e os erros de tipo da fase semântica.

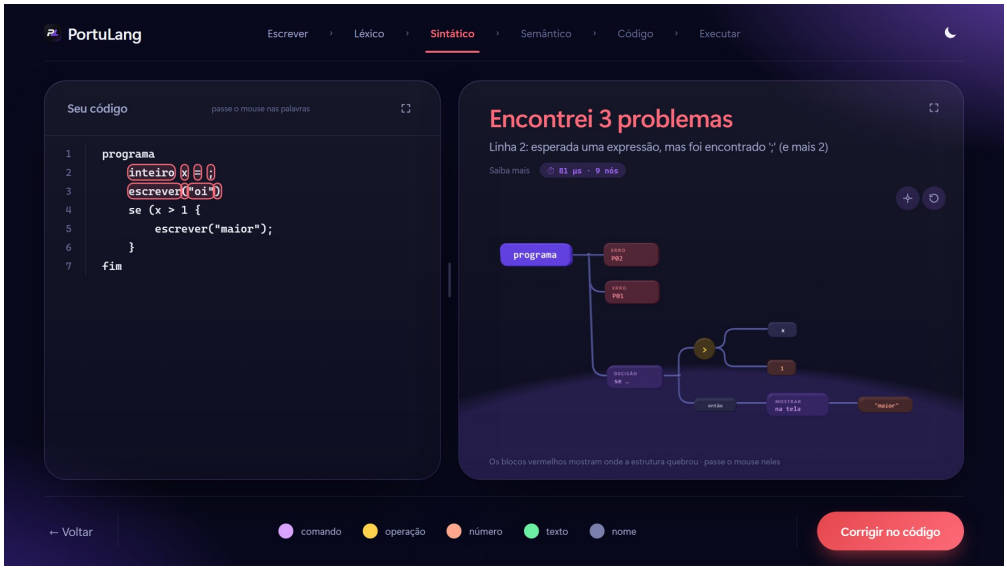


Figura 7. Erro sintático: os blocos vermelhos mostram onde a estrutura quebrou.



Figura 8. Erros semânticos marcados no código e na árvore.

12. Testes

Os testes automatizados usam apenas a biblioteca padrão (`unittest`): **52 testes**, executados com `python -m unittest discover -s tests`. Dois grupos comparam implementações diferentes entre si: o Python gerado e a máquina virtual precisam dar exatamente a mesma saída (e o mesmo erro) que o interpretador, nos exemplos e em casos especiais (curto-circuito, escopos, divisão inteira negativa, nomes reservados do Python, entrada inválida).

Grupo	Testes	O que verifica
`TesteLexer`	6	tokens, posições, literais, erros léxicos
`TesteParser`	11	precedência, estruturas, erros e recuperação, faixas de tokens da AST
`TesteSemantico`	6	regras S01 a S09 e tabela de símbolos
`TesteInterpretador`	15	execução, escopos, entrada/saída, erros de execução

Grupo	Testes	O que verifica
`TesteExemplos`	5	programas de exemplo corretos e com erro propositais
`TesteDelimitadoresAbertos`	2	erro P05 (parêntese ou chave aberto)
`TesteCodigoIntermediario`	5	máquina virtual = interpretador (saída e erros)
`TesteTranspilador`	2	Python gerado = interpretador (saída e erros)
`Total`	52	

Além dos testes automáticos, o projeto traz exemplos prontos: quatro programas corretos (01_idade, 02_media, 03_fatorial, 04_logico) e quatro com um tipo de erro cada (léxico, sintático, semântico e de execução), usados na demonstração.

13. Como obter e executar

O código-fonte está em **github.com/mateusartur27/portulang** (repositório público).

Objetivo	Como
Usar o Studio no Windows, sem instalar nada	`Extrair o PortuLang-windows.zip e abrir o PortuLang.exe (abre numa janela própria)`
Usar o Studio pelo navegador (qualquer sistema)	`python -m studio (abre http://127.0.0.1:8765) ou abrir_studio.bat`
Executar um programa pela linha de comando	`python -m portulang exemplos/01_idade.pl`
Ver as fases pela linha de comando	`python -m portulang arq.pl --tokens | --ast | --simbolos | --json`
Rodar os testes	`python -m unittest discover -s tests`
Gerar o executável e o zip	`powershell -File build_exe.ps1 (precisa de Python 3.11+)`

Requisitos: Python 3.11 ou mais novo para rodar a partir do código-fonte (sem dependências externas); o executável para Windows já leva o Python embutido e usa o WebView2, que vem no Windows 11. Como o executável ainda não tem assinatura digital, o Windows pode mostrar um aviso na primeira vez (“Mais informações”, depois “Executar assim mesmo”).

14. Exemplos

14.1 Programa com entrada de dados (02_media)

```
programa
    texto nome;
    decimal nota1;
    decimal nota2;

    escrever("Qual o seu nome? ");
    ler(nome);
    escrever("Nota 1: ");
    ler(nota1);
    escrever("Nota 2: ");
    ler(nota2);

    decimal media = (nota1 + nota2) / 2;

    se (media >= 7) {
        escrever(nome, " foi aprovado(a) com ", media);
    } senao se (media >= 5) {
        escrever(nome, " esta de recuperacao com ", media);
    } senao {
        escrever(nome, " foi reprovado(a) com ", media);
    }
fim
```

14.2 Programa com erros semânticos

```
programa
    inteiro idade = "vinte";   // S03: texto em inteiro
    total = 10;                // S01: 'total' não foi declarado
    inteiro idade = 3;         // S02: 'idade' já foi declarada
    se (idade) {               // S07: condição não é lógica
        escrever("oi");
    }
fim
```

14.3 Repetição (03_fatorial)

```
// fatorial de um numero lido
programa
    inteiro n;
    escrever("Digite um numero: ");
    ler(n);

    inteiro resultado = 1;
    inteiro i = 2;
    enquanto (i <= n) {
        resultado = resultado * i;
        i = i + 1;
    }
    escrever(n, "! = ", resultado);
fim
```

15. Entregas do projeto

A tabela relaciona cada entrega do cronograma da disciplina ao que foi produzido e onde encontrá-lo.

Sem.	Entrega	Situação	Onde está
1	Especificação inicial	Concluída	Seções 1 a 3 e 6 a 7 deste documento
1	Tabela de tokens	Concluída	Seção 4
2	Gramática em EBNF	Concluída	Seção 5

Sem.	Entrega	Situação	Onde está
2	Lexer funcional	Concluída	portulang/lexer.py, tokens.py; testes TesteLexer
3	Parser e AST	Concluída	portulang/parser.py, ast_nodes.py; testes TesteParser
3	Validador semântico	Concluída	portulang/semantico.py; testes TesteSemantico
4	Execução dos programas (tradutor)	Concluída	transpilador.py (Python), interpretador.py, codigo.py
4	Testes, documentação e apresentação (projeto final)	Concluída	52 testes; este documento; README; apresentação em slides; Studio

Critérios de avaliação. Especificação e gramática: seções 1 a 7. Analisadores léxico, sintático e semântico: seções 4, 5, 7, 8 e 9. Funcionamento do interpretador: seção 10. Testes e tratamento de erros: seções 8 e 12. Documentação técnica: este documento e o README do repositório. Apresentação e demonstração: apresentação em slides e PortuLang Studio.

16. Limitações e próximos passos

- A linguagem não tem funções, vetores nem comando de parada (break); é pequena de propósito.
- Não há otimização nem geração de código de máquina: a execução depende do Python.
- O executável para Windows não tem assinatura digital; um instalador e versões para outros sistemas ficam como evolução.
- Possíveis extensões: funções e vetores, mais tipos, otimizações simples (dobra de constantes) na geração de código.

Autores: Mateus Artur, Rafael Araújo e Otávio de Pádua.